

SAN JOSÉ STATE UNIVERSITY · CMPE 172 ENTERPRISE SOFTWARE PLATFORMS

TutorBook

Term Project, Milestone 1 Report
Requirements, Design & Skeleton

Student	Samuel Dinkayehu
Scenario	Tutoring center appointment booking
Stack	Java 17+ · Spring Boot 4 · SQL over JDBC (no ORM) · Thymeleaf · H2
GitHub	https://github.com/samuel1gg/cmpe172-tutorbook
Date	September 24, 2026

Contents

1. Proposal: scenario, roles, features, entities, architecture
2. Database design: ER diagram, relational schema, relationships, double-booking guard, justification
3. Wireframes
4. Running skeleton: structure, endpoints, schema.sql / seed.sql, screenshots
5. Request flow through the layers
6. Page Controller vs Front Controller
7. How to run and testing

1. Proposal

1.1 Chosen scenario

TutorBook is an appointment-booking system for a campus **tutoring center**. Tutors publish the times they are available for a particular kind of session, such as a 60-minute Calculus session or a 30-minute Essay Review. Students browse those open times, filter them, and book one. The system's main guarantee is that **a time slot can never be booked by two students at once**. Students can later view and cancel their appointments, and tutors can see who has booked with them.

1.2 User roles

Role	Who	What they can do
Customer	Student	Log in, browse/filter/paginate open slots, book a slot, view upcoming and past appointments, cancel their own appointment.
Provider	Tutor	Log in, create and remove availability slots for a service, view appointments booked with them.
Admin	Center staff	Maintains the service catalog and provider profiles. Stored as a role value; the admin screens are out of scope for now.

1.3 Core features

#	Feature	Role	Milestone
1	Customer log in	Customer	M2
2	Browse, filter (provider, service, date) and paginate available slots using SQL <code>LIMIT / OFFSET</code>	Customer	M2 (read path already in M1)
3	Book a slot for a chosen service	Customer	M2
4	View my appointments (upcoming + history)	Customer	M2
5	Cancel my appointment (owner-only)	Customer	M2

6	Provider log in	Provider	M2
7	Provider: create / remove availability slots (tied to a service)	Provider	M2
8	Provider: view appointments booked with me	Provider	M2
9	Send a confirmation via a mock external notification service on book/cancel	System	M3
10	Logging, health endpoint, and at least one metric (e.g. <code>bookings.created</code> counter)	System	M3
11	AI agents: a conversational booking agent and an autonomous agent (e.g. one that reminds students or fills cancelled slots)	Customer / System	M4
12	Grounded Q&A (retrieval-augmented) over a small knowledge base (center policies, tutor bios, FAQs)	Customer	M4

Milestone 1 delivers the skeleton that these features will be built on. It already has the database, the layered architecture, and the read paths for the home page and the available-slot list, including filtering and pagination.

1.4 Main entities

Entity	Meaning
users	Everyone who can log in (students, tutors and admins), with a <code>role</code> column.
providers	The tutor profile (display name, specialty, bio) attached to a user whose role is PROVIDER.
services	The catalog of session types: name, description, duration, price.
availability_slots	A block of time a provider opens for one service.
appointments	A customer's booking of one slot, with status BOOKED or CANCELLED.

1.5 Block diagram

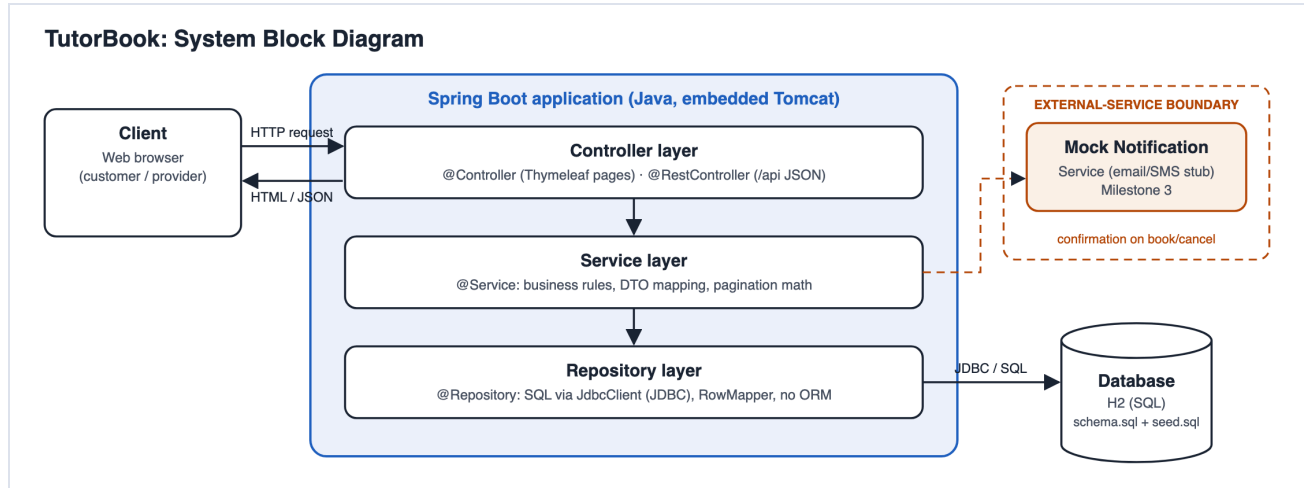


Figure 1. Client → Spring Boot application (Controller → Service → Repository) → SQL database over JDBC, with the mock external notification service outside the system boundary.

1.6 Technology statement

TutorBook is implemented in **Java** with **Spring Boot**. All data access is written as **plain SQL executed through JDBC** (Spring's `JdbcTemplate`, which sits on top of `JdbcTemplate`). The project uses **no ORM**: no JPA, Hibernate or Spring Data repositories. Every query is hand-written, and every result row is mapped to a Java record by an explicit `RowMapper`.

This project realizes classic enterprise standards on the modern stack: **Spring Boot stands in for J2EE** (container-managed components, dependency injection and transactions), and **REST over HTTP stands in for CORBA and distributed objects** as the way remote clients and services communicate.

2. Database Design

2.1 ER diagram

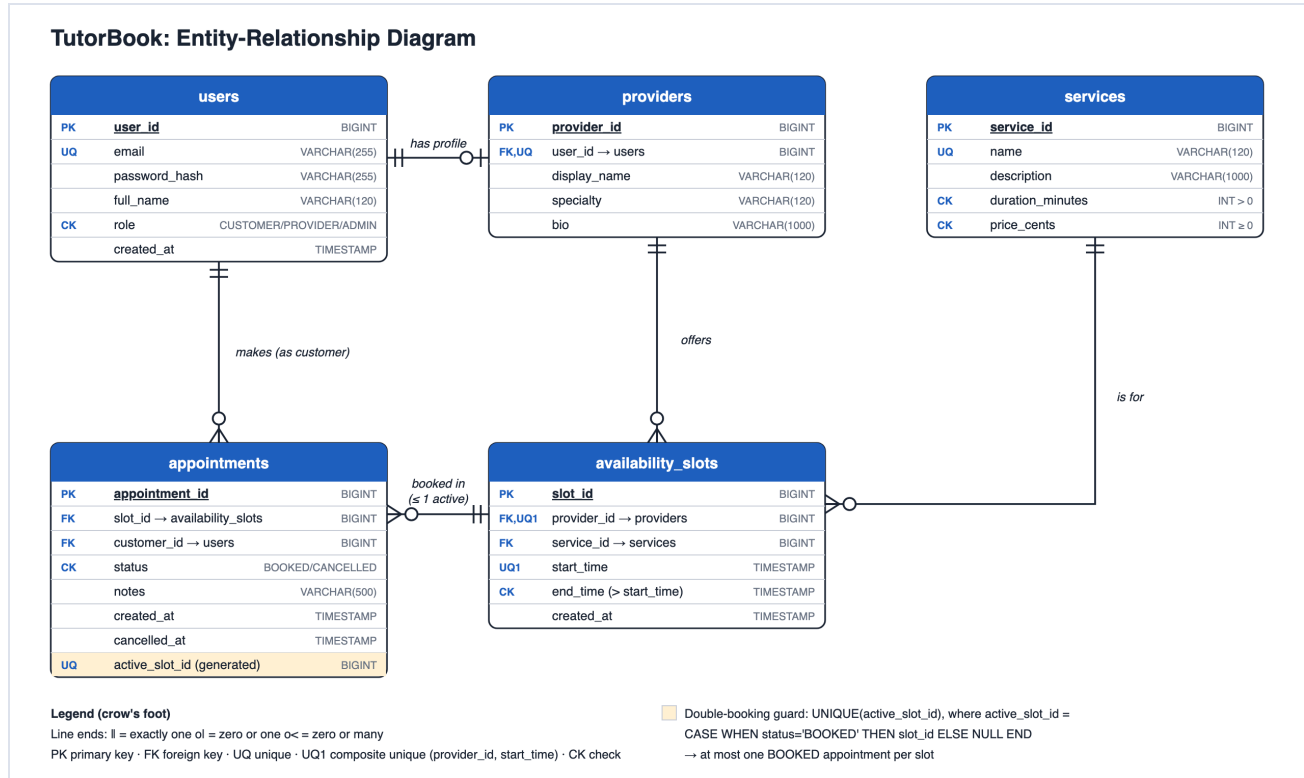


Figure 2. TutorBook ER diagram in crow's-foot notation. The highlighted column implements the double-booking guard.

2.2 Relationships

Relationship	Cardinality	Implemented by	Explanation
users <i>has profile</i> providers	1 : 0..1	providers.user_id FK + UNIQUE	A user has at most one tutor profile, and every profile belongs to exactly one user. The UNIQUE constraint on the FK turns a normal 1:N into 1:1. Customers simply have no providers row.
providers <i>offers</i> availability_slots	1 : 0..N	availability_slots.provider_id FK (NOT NULL)	A tutor can open many slots, and each slot belongs to exactly one tutor. ON DELETE CASCADE removes a tutor's slots if the tutor is removed.
services <i>is for</i> availability_slots	1 : 0..N	availability_slots.service_id FK (NOT NULL)	Each slot is tied to one service (feature 7), and a service can be offered in many slots. There is no cascade, because a service still in use cannot be deleted.

availability_slots <i>booked in</i> appointments	1 : 0..N, at most 1 active	appointments.slot_id FK + UNIQUE(active_slot_id)	Over time a slot can collect several appointment rows (booked, cancelled, booked again), but at most one of them may be BOOKED at any moment.
users <i>makes</i> appointments	1 : 0..N	appointments.customer_id FK	A customer can have many appointments (upcoming and history), and each appointment belongs to one customer. This is the ownership used for "cancel my appointment (owner-only)".

Why there is no direct providers ↔ services table: which services a tutor offers can be derived from their slots (SELECT DISTINCT service_id FROM availability_slots WHERE provider_id = ?). A separate provider_services table would duplicate that information. Keeping the five required entities also keeps the model minimal.

Relational schema (notation)

```

users(user_id, email, password_hash, full_name, role, created_at)
    UNIQUE(email) · CHECK(role IN ('CUSTOMER','PROVIDER','ADMIN'))

providers(provider_id, user_id, display_name, specialty, bio)
    FK user_id → users(user_id) ON DELETE CASCADE · UNIQUE(user_id)

services(service_id, name, description, duration_minutes, price_cents)
    UNIQUE(name) · CHECK(duration_minutes > 0) · CHECK(price_cents ≥ 0)

availability_slots(slot_id, provider_id, service_id, start_time, end_time,
    created_at)
    FK provider_id → providers(provider_id) ON DELETE CASCADE · FK service_id →
    services(service_id)
    CHECK(end_time > start_time) · UNIQUE(provider_id, start_time)

appointments(appointment_id, slot_id, customer_id, status, notes, created_at,
    cancelled_at, active_slot_id)
    FK slot_id → availability_slots(slot_id) · FK customer_id → users(user_id)
    CHECK(status IN ('BOOKED','CANCELLED'))
    active_slot_id GENERATED ALWAYS AS (CASE WHEN status='BOOKED' THEN slot_id ELSE
    NULL END)
    UNIQUE(active_slot_id) ← double-booking guard

```

Legend: underlined = primary key, *italic* = foreign key. Every primary key is a surrogate **BIGINT** identity.

Column detail

Table	Column	Type	Null?	Key / constraint
users	user_id	BIGINT identity	NO	PK
	email	VARCHAR(255)	NO	UNIQUE (login name)
	password_hash	VARCHAR(255)	NO	stores a hash, never plain text (BCrypt in M2)
	full_name	VARCHAR(120)	NO	
	role	VARCHAR(20)	NO	CHECK CUSTOMER / PROVIDER / ADMIN
	created_at	TIMESTAMP	NO	DEFAULT CURRENT_TIMESTAMP
providers	provider_id	BIGINT identity	NO	PK
	user_id	BIGINT	NO	FK → users, UNIQUE (1:1)
	display_name	VARCHAR(120)	NO	
	specialty	VARCHAR(120)	NO	
	bio	VARCHAR(1000)	YES	
services	service_id	BIGINT identity	NO	PK
	name	VARCHAR(120)	NO	UNIQUE
	description	VARCHAR(1000)	YES	
	duration_minutes	INT	NO	CHECK > 0
	price_cents	INT	NO	CHECK ≥ 0 (integer cents, no float rounding)
availability_slots	slot_id	BIGINT identity	NO	PK
	provider_id	BIGINT	NO	FK → providers; part of UNIQUE(provider_id, start_time)
	service_id	BIGINT	NO	FK → services
	start_time	TIMESTAMP	NO	indexed; part of UNIQUE(provider_id, start_time)
	end_time	TIMESTAMP	NO	CHECK end_time > start_time
	created_at	TIMESTAMP	NO	DEFAULT CURRENT_TIMESTAMP

appointments	appointment_id	BIGINT identity	NO	PK
	slot_id	BIGINT	NO	FK → availability_slots
	customer_id	BIGINT	NO	FK → users; indexed
	status	VARCHAR(20)	NO	CHECK BOOKED / CANCELLED, DEFAULT 'BOOKED'
	notes	VARCHAR(500)	YES	
	created_at	TIMESTAMP	NO	DEFAULT CURRENT_TIMESTAMP
	cancelled_at	TIMESTAMP	YES	set when cancelled
	active_slot_id	BIGINT (generated)	YES	UNIQUE: the double-booking guard

2.3 The double-booking guard

```
CREATE TABLE appointments (
  appointment_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  slot_id        BIGINT NOT NULL,
  customer_id    BIGINT NOT NULL,
  status         VARCHAR(20) DEFAULT 'BOOKED' NOT NULL,
  ...
  active_slot_id BIGINT GENERATED ALWAYS AS
                  (CASE WHEN status = 'BOOKED' THEN slot_id ELSE NULL END),
  ...
  CONSTRAINT uq_appt_active_slot UNIQUE (active_slot_id)
);
```

How it works. `active_slot_id` is a *generated* column. The database computes it and the application never writes it. While an appointment is `BOOKED` it holds the slot's id, and once the appointment is `CANCELLED` it becomes `NULL`. Because there is a **UNIQUE constraint on `active_slot_id`**, the database will not store two `BOOKED` appointments with the same slot id. SQL UNIQUE constraints treat NULLs as distinct, so any number of cancelled appointments can reference the same slot.

Why it is done in the database and not only in Java. Suppose two students click "Book" on the same slot at the same instant. A check in Java such as "SELECT, see it's free, then INSERT" has a race condition: both requests can pass the SELECT before either INSERT runs. The UNIQUE index is enforced atomically by the database engine, so exactly one INSERT succeeds and the other fails with a unique-constraint violation. Spring translates that into `DuplicateKeyException`, and in Milestone 2 the service layer will turn it into a friendly "this slot was just taken" message.

Why not simply `UNIQUE(slot_id)`? That would also stop double-booking, but a slot could never be booked again after a cancellation unless the appointment row were deleted, and deleting it would lose the customer's history (feature 4). The generated-column approach keeps both the guarantee and the history.

A second guard on the provider side: `UNIQUE(provider_id, start_time)` on `availability_slots` stops a tutor from publishing two slots that start at the same time, so a tutor cannot double-book

themselves either.

Proof: `DoubleBookingGuardTest` inserts a `BOOKED` appointment, attempts a second one for the same slot, and asserts that a `DuplicateKeyException` is thrown. It also shows that after a cancellation the same slot can be booked again.

2.4 Design justification

Cardinality. The cardinalities in 2.2 come directly from the business rules. A tutor has one profile (1:1). A tutor offers many slots, and a slot belongs to one tutor (1:N). A slot is for one service (N:1). A student has many appointments (1:N). A slot can have at most one active appointment, which is 1:0..1 at any moment and 1:N over its history. The one-to-one and the "at most one active" rules cannot be expressed with foreign keys alone, so both are enforced with `UNIQUE` constraints.

Weak entities. In the conceptual model, `availability_slots` is *existence-dependent* on `providers` : a slot means nothing without its tutor. It could be modeled as a weak entity whose partial key is `start_time` , identified by `(provider_id, start_time)` . Likewise, `appointments` is an associative entity that links a customer to a slot. In the physical schema I give both tables a **surrogate primary key** `(slot_id, appointment_id)` because:

- foreign keys and URLs such as `/slots/7/book` stay short and stable;
- the natural key is still enforced as `UNIQUE(provider_id, start_time)` , so no identifying information is lost;
- the weak-entity dependency is kept with `NOT NULL` foreign keys and `ON DELETE CASCADE` from providers to slots.

Constraints. Each constraint protects a specific business rule at the lowest layer, so no code path can bypass it:

- `NOT NULL` on every required column;
- `CHECK` constraints for enumerations (role, status) and sanity rules (`end_time > start_time` , positive duration, non-negative price);
- `UNIQUE` constraints for identity (email, service name) and the booking rules above;
- foreign keys for referential integrity, with `CASCADE` only where child rows cannot exist without the parent.

Money is stored as integer **cents** to avoid floating-point rounding. Indexes on `availability_slots(start_time)` , `(service_id)` and `appointments(customer_id)` support the most frequent queries: browsing slots and "my appointments".

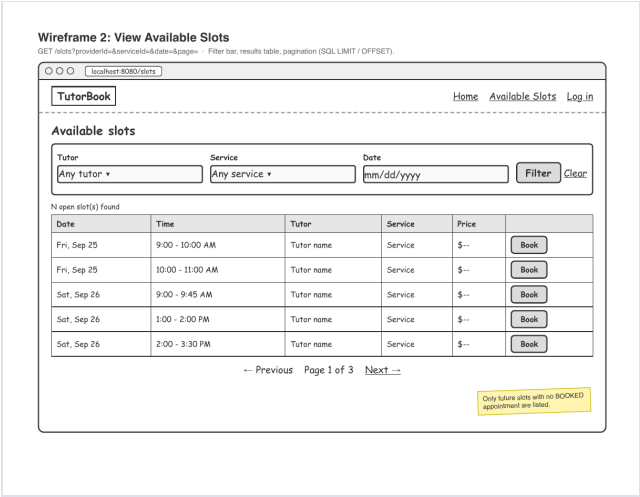
Normalization. Every non-key attribute depends on the whole key and nothing but the key, so all tables are in 3NF. For example, a service's price lives only in `services` and is joined when needed, never copied into slots.

3. Wireframes

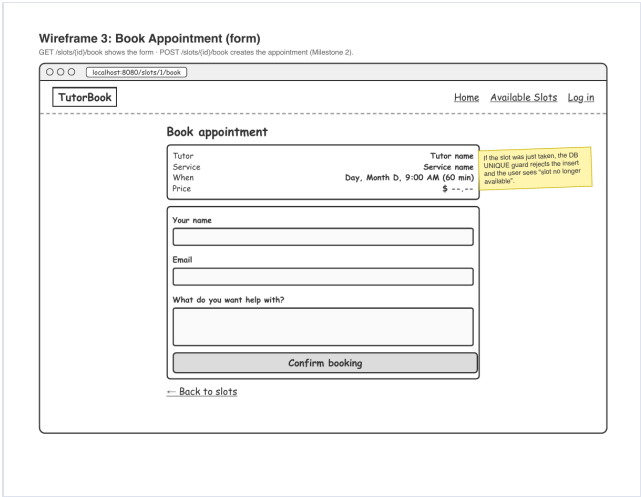
The four required screens are shown as low-fidelity wireframes. The complete set is also submitted separately as Wireframes.pdf .



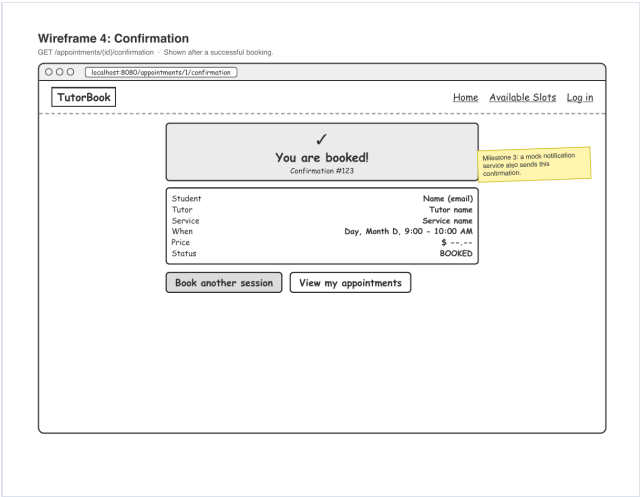
W1. Home



W2. View available slots



W3. Book appointment (form)



W4. Confirmation

Navigation flow: Home → (Browse / See open slots / Find a time) → Available Slots (filters + paging) → Book → Confirmation.

4. Running Skeleton

4.1 Project structure

```
src/main/java/edu/sjsu/cmpe172/tutorbook/
├── TutorBookApplication.java      @SpringBootApplication entry point
├── controller/                   WEB LAYER
│   ├── HomeController            GET /
│   ├── SlotController           GET /slots, GET /slots/{id}/book
│   ├── AppointmentController    GET /appointments/{id}/confirmation
│   └── CatalogApiController      GET /api/slots, /api/providers, /api/services
├── (JSON)
├── service/                     BUSINESS LAYER
│   ├── CatalogService           home page + provider/service lists
│   ├── SlotService              paging math, not-found rule, DTO mapping
│   └── AppointmentService        appointment lookup (booking logic lands in M2)
├── repository/                 DATA LAYER (JDBC, hand-written SQL)
│   ├── ProviderRepository, ServiceOfferingRepository
│   ├── SlotRepository           available-slot query with filters + LIMIT/OFFSET
│   └── AppointmentRepository    find / insertBooked / cancel
├── model/                      records mirroring DB rows (Provider, SlotDetails, ...)
├── dto/                        records handed to views/JSON (HomeDto, SlotDto, PageDto, ...)
├── exception/NotFoundException  → HTTP 404
└── src/main/resources/
    ├── schema.sql seed.sql      loaded on startup
    ├── templates/             home, slots, book, confirmation (Thymeleaf)
    └── application.properties
```

Dependencies only point downward: controllers use services, services use repositories, and only repositories contain SQL. Each class receives its dependencies through **constructor injection**, and Spring creates the objects (inversion of control). **Model** records mirror database rows. **DTO** records are what leaves the service layer: they hide internal columns such as `user_id` and `password_hash` and add presentation-ready values such as the formatted price.

4.2 schema.sql and seed.sql, loaded on startup

```
spring.datasource.url=jdbc:h2:mem:tutorbook;DB_CLOSE_DELAY=-1
spring.sql.init.mode=always
spring.sql.init.schema-locations=classpath:schema.sql
spring.sql.init.data-locations=classpath:seed.sql
```

On startup Spring Boot runs `schema.sql`, which drops and recreates all five tables with every constraint and index, and then `seed.sql`. The seed creates 8 users (3 customers, 4 tutors, 1 admin), 4 provider profiles, 5 services, 14 slots and 3 appointments. One of those appointments is **BOOKED**, which hides its slot from the list. Another is **CANCELLED**, which leaves its slot bookable again and

demonstrates the guard. Slot times are computed relative to `CURRENT_DATE` , so the demo always shows upcoming slots.

4.3 Endpoints (all read from the database and return DTOs)

Method & URL	Controller → Service	Returns
GET /	HomeController → CatalogService.getHome()	View <code>home</code> with <code>HomeDto</code>
GET /slots? providerId&serviceId&date&page&size	SlotController → SlotService.findAvailableSlots()	View <code>slots</code> with <code>PageDto<SlotDto></code>
GET /slots/{id}/book	SlotController → SlotService.getAvailableSlot()	View <code>book</code> with <code>SlotDto</code> (404 if taken)
GET /appointments/{id}/confirmation	AppointmentController → AppointmentService	View <code>confirmation</code> with <code>AppointmentDto</code>
GET /api/slots, /api/slots/{id}	CatalogApiController → SlotService	JSON <code>PageDto<SlotDto></code> / <code>SlotDto</code>
GET /api/providers, /api/services	CatalogApiController → CatalogService	JSON lists of DTOs
GET /actuator/health	Spring Boot Actuator	<code>{"status":"UP"}</code>

4.4 Screenshots of the running application

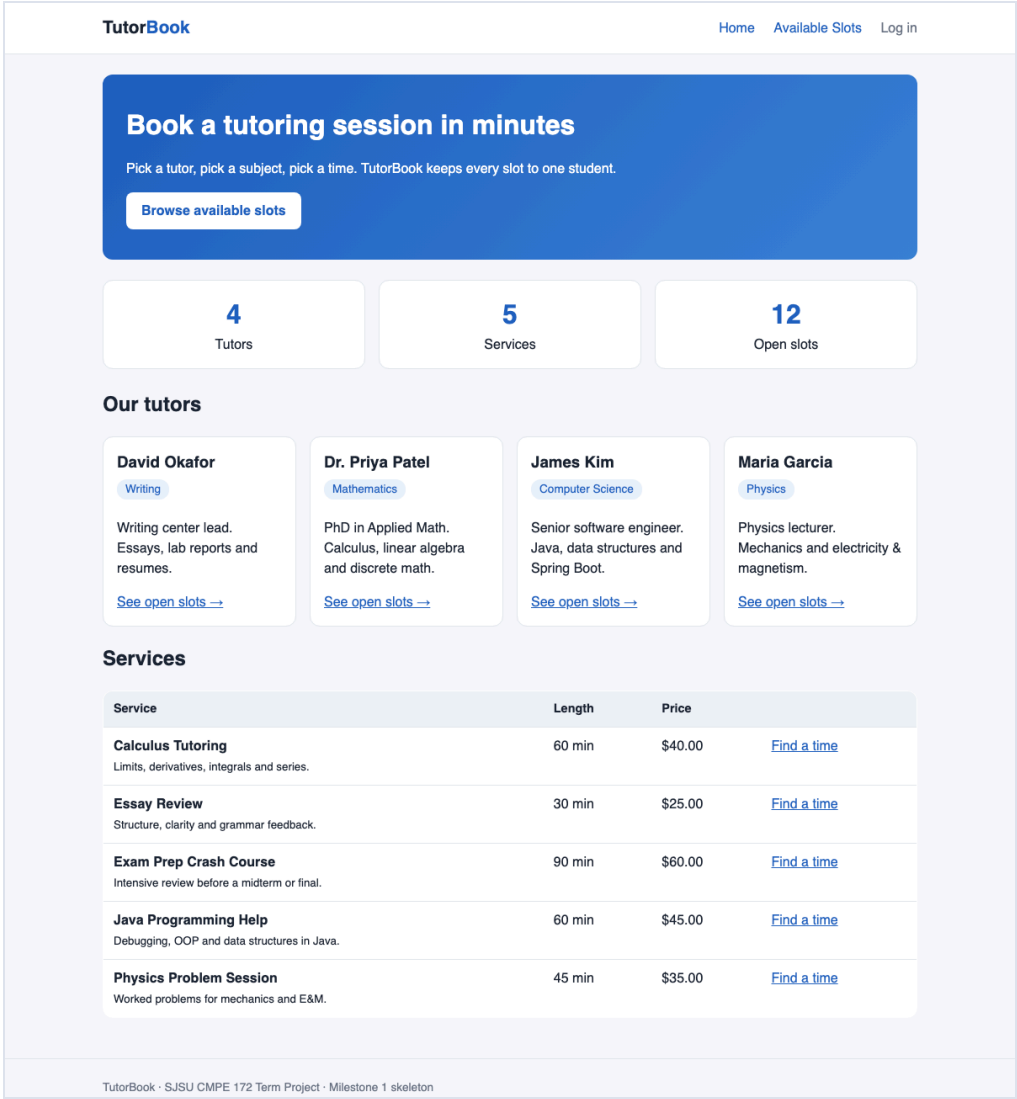


Figure 3. GET /: every number, tutor and service comes from the database.

TutorBook
Home Available Slots Log in

Available slots

Tutor
Any tutor

Service
Any service

Date
mm/dd/yyyy

Filter

Clear

12 open slot(s) found

Date	Time	Tutor	Service	Price	
Fri, Sep 25	9:00 AM - 10:00 AM	Dr. Priya Patel	Calculus Tutoring	\$40.00	Book
Fri, Sep 25	10:00 AM - 11:00 AM	James Kim	Java Programming Help	\$45.00	Book
Sat, Sep 26	9:00 AM - 9:45 AM	Maria Garcia	Physics Problem Session	\$35.00	Book
Sat, Sep 26	1:00 PM - 2:00 PM	James Kim	Java Programming Help	\$45.00	Book
Sat, Sep 26	2:00 PM - 3:30 PM	Dr. Priya Patel	Exam Prep Crash Course	\$60.00	Book

Page 1 of 3
Next →

Figure 4. GET /slots: 12 open slots, 5 per page (LIMIT 5 OFFSET 0). Slot 2 is booked, so it is not listed.

TutorBook
Home Available Slots Log in

Book appointment

Tutor
Dr. Priya Patel

Service
Calculus Tutoring

When
Friday, September 25, 2026 9:00 AM (60 min)

Price
\$40.00

Your name
Filled in from your login (Milestone 2)

Email
Filled in from your login (Milestone 2)

What do you want help with?
e.g. integration by parts, chapter 7

Confirm booking

Submitting a booking is implemented in Milestone 2 (feature 3). This page already reads the slot from the database.

[← Back to slots](#)

TutorBook - SJSU CMPE 172 Term Project - Milestone 1 skeleton

Figure 5. GET /slots/1/book

TutorBook
Home Available Slots Log in

You are booked!
Confirmation #1

Student
Alice Nguyen (alice@student.sjsu.edu)

Tutor
Dr. Priya Patel

Service
Calculus Tutoring

When
Friday, September 25, 2026 11:00 AM - 12:00 PM

Price
\$40.00

Notes
Need help with integration by parts.

Status
BOOKED

Book another session

TutorBook - SJSU CMPE 172 Term Project - Milestone 1 skeleton

Figure 6. GET /appointments/1/confirmation

5. Request Flow Through the Layers

The following traces `GET /slots?serviceld=2&page=0&size=5` from the browser to the database and back.

1. **Browser** → **embedded Tomcat**. The HTTP request arrives at the servlet container embedded in the Spring Boot jar.
2. **DispatcherServlet**. Spring MVC's single servlet looks up the handler mapped to `GET /slots` and finds `SlotController.listSlots`.
3. **Controller layer**: `SlotController.listSlots(SlotSearchCriteria criteria, Model model)`. Spring binds the query parameters into the `SlotSearchCriteria` record (`serviceld=2, page=0, size=5`). The controller contains no logic. It asks `SlotService` for the page of slots and `CatalogService` for the filter drop-down lists, puts them in the `Model`, and returns the view name "slots".
4. **Service layer**: `SlotService.findAvailableSlots(criteria)` applies the business rules. It defaults and clamps the page size to 1–50, computes `offset = page × size`, calls the repository twice (once for the rows, once for the total count), converts each `SlotDetails` model into a `SlotDto` (formatting cents as "\$45.00"), and wraps everything in `PageDto`, which calculates `totalPages`, `hasNext` and `hasPrevious`.
5. **Repository layer**: `SlotRepository.findAvailable(criteria, 5, 0)` builds the SQL. It joins slots, providers and services; keeps only future slots with no BOOKED appointment (`NOT EXISTS ... a.active_slot_id = s.slot_id`); appends `AND s.service_id = :serviceld` because that filter is present; and adds `ORDER BY s.start_time LIMIT :limit OFFSET :offset`. Values are passed as **named parameters**, never concatenated into the string, which prevents SQL injection.
6. **JDBC** → **database**. `JdbcClient` borrows a connection from the HikariCP pool, prepares the statement, executes it on H2, and hands each `ResultSet` row to the row mapper, which builds a `SlotDetails` record.
7. **Back up the stack**. The records return to the service (which turns them into DTOs), then to the controller (`Model`). The **Thymeleaf** view `slots.html` renders the HTML table and pagination links, and the response goes back to the browser.

```
Browser -GET /slots-> DispatcherServlet -> SlotController -> SlotService ->
SlotRepository -SQL-> H2
Browser <-HTML- Thymeleaf(slots.html) <- Model <- PageDto<SlotDto> <-
List<SlotDetails> <- ResultSet
```

Error path: `GET /slots/2/book` for a slot that is already booked. The repository returns `Optional.empty()`, the service throws `NotFoundException` (annotated `@ResponseStatus(NOT_FOUND)`), and Spring renders `error/404.html`. The controller needs no try/catch.

Same service, different view: `GET /api/slots` goes through `CatalogApiController`, which calls **the same** `SlotService.findAvailableSlots` method. The only difference is that `@RestController` serializes the

`PageDto` to JSON instead of rendering a template. This shows the service and repository layers do not depend on how the result is presented.

6. Page Controller vs Front Controller

	Page Controller	Front Controller
Idea	One controller object per page or action. Each request for a page goes to the controller that owns that page, which prepares data and chooses that page's view.	A single entry-point handler receives <i>every</i> request, performs common work (routing, security, content negotiation), and dispatches to a command or handler.
Typical use	Server-rendered, multi-page web apps	Single-page apps (SPA) that talk to a JSON API; frameworks with one central router
In Spring	<code>@Controller</code> classes returning Thymeleaf view names	Spring's <code>DispatcherServlet</code> is the front controller for all of Spring MVC

My choice: Page Controller, with server-rendered Thymeleaf. Each page in the wireframes has its own controller method. `HomeController` owns `/`, `SlotController` owns `/slots` and the booking form, and `AppointmentController` owns the confirmation page. Each one fills a `Model` and returns its own view. I chose this because:

- the application is a small multi-page site where each screen maps naturally to one controller method, which matches the wireframes one-to-one;
- server-side rendering needs no separate JavaScript build or client-side state, and it keeps the focus of this course on the server tiers (service, JDBC, database);
- forms (booking, cancel, login) work with plain HTML POSTs and Spring Security's session login in Milestone 2.

Spring MVC always has the `DispatcherServlet` sitting in front, so technically a front controller is always present at the framework level. The design choice is what sits behind it. My application code follows the Page Controller pattern. If the UI later became an SPA, the browser would call only the `/api/**` JSON endpoints, and the `DispatcherServlet` would act as the application's Front Controller that routes every API call. The service and repository layers would not change, and `CatalogApiController` already shows this.

7. How to Run and Testing

```
git clone https://github.com/samuel1gg/cmpe172-tutorbook.git
cd cmpe172-tutorbook
./mvnw spring-boot:run      # then open http://localhost:8080
./mvnw test                  # 11 tests
```

The only requirement is JDK 17 or newer. The Maven wrapper downloads Maven, and H2 runs in memory, so no database installation is needed. The H2 console at `/h2-console` (JDBC URL `jdbc:h2:mem:tutorbook`, user `sa`) lets you inspect the tables.

Test class	What it proves
DoubleBookingGuardTest (4 tests)	A second BOOKED appointment for the same slot throws <code>DuplicateKeyException</code> ; a booked slot disappears from the available list; a cancelled slot can be booked again; the seed data behaves as documented.
WebEndpointsTest (7 tests)	<code>GET /</code> and <code>GET /slots</code> render database data; <code>/api/slots</code> paginates correctly (12 items, size 5 → 3 pages) and filters by provider; a booked slot returns 404; the booking and confirmation pages render; <code>/actuator/health</code> is UP.

All 11 tests pass.